B22 Evaluating LLM systems

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B22 Evaluating LLM systems 3
B22-L1 Evaluate retrieval, recall at k and MRR on a gold set 3
B22-L2 Evaluate answers, exact checks, token F1 and LLM judges 8
B22-L3 Regression evals and release gates for prompt changes 14
Track B best-practice checklist 20
Appendix: solutions and answer keys 21

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B22 Evaluating LLM systems

Roadmap: Months 13 to 15: LLMs. Retrieval metrics, answer checks, LLM judges and regression gates for prompt changes.

B22-L1 Evaluate retrieval, recall at k and MRR on a gold set

Concept

A RAG answer can fail in two places: retrieval missed the passage, or the model misused it. Measure retrieval on its own first. It is cheap and exact, and needs no model call.

Build a gold set. Write questions the way users ask them, and record for each one the IDs of the sections that answer it. Take questions from real requests where you can: users phrase things in ways you do not expect. Mark relevance by section, not by chunk, so a new chunk size does not invalidate the labels. Keep the set in version control, and add every question that failed in use.

Two metrics.

- **Recall at k** is the share of a question's relevant sections found in the top k. With one relevant section it is 1 or 0, and its mean over questions is the hit rate.
- **Reciprocal rank** is 1 divided by the rank of the first relevant result, counting from 1, or 0 when none appears. Its mean over questions is the **mean reciprocal rank** (MRR). It rewards putting the answer first, not only somewhere in the list.

```python
def recall_at_k(ranked: list[str], relevant: set[str], k: int) -> float:
    return len(set(ranked[:k]) & relevant) / len(relevant)


def reciprocal_rank(ranked: list[str], relevant: set[str]) -> float:
    for rank, item in enumerate(ranked, start=1):
        if item in relevant:
            return 1 / rank
    return 0.0
```

`&` intersects two sets. Rank sections, not chunks: when several chunks of one section come back, keep the section at its first position with `list(dict.fromkeys(section_ids))`, which drops repeats and keeps order.

Compare settings on the same set. Chunk size, k, the keyword ranking and the filters all change retrieval. Run each on the same gold set and keep the best. Report intervals: 45 hits out of 50 has a 95% Wilson interval of 78.6% to 95.7% (lesson B8-L2), so one question more or less proves nothing.

Why it matters

A team spent two weeks rewriting its answer prompt because a third of answers were wrong. A retrieval check on 60 gold questions then showed recall at 5 of 0.68: for about a third of the questions, the right section never reached the prompt. No prompt could fix that. Adding a keyword ranking raised recall at 5 to 0.93, and most wrong answers went with it.

Worked example: weak code

```
1 def check_retrieval(search):
1     results = search("What is the weight cap?")
3     print(results[:5])
```

1. **Line 2.** One question, read by eye. No gold labels, no metric, nothing to compare next time.

Worked example: strong code

```
1  from collections.abc import Callable
2
3
4  def recall_at_k(ranked: list[str], relevant: set[str], k: int) -> float:
5      """Share of the relevant IDs found in the first k of a ranking.
6
7      Raises:
8          ValueError: If relevant is empty or k is below 1.
9      """
1      if not relevant:
11         raise ValueError("relevant is empty: the question has no gold
       answer")
12     if k < 1:
13         raise ValueError(f"k must be at least 1, got {k}")
2      return len(set(ranked[:k]) & relevant) / len(relevant)
15
16
17 def reciprocal_rank(ranked: list[str], relevant: set[str]) -> float:
18     """1 / rank of the first relevant ID, counting from 1, or 0.0 if
       none."""
3      for rank, item in enumerate(ranked, start=1):
20         if item in relevant:
21             return 1 / rank
22     return 0.0
23
24
```

```python
def retrieval_scores(
    search: Callable[[str], list[str]], gold: list[tuple[str, set[str]]], k: int
) -> dict[str, float]:
    """Mean recall at k and MRR of a section-level search over gold questions.

    Args:
        search: Returns section IDs for a question, best first, possibly repeated.
        gold: (question, relevant section IDs) pairs.
        k: Cut-off for recall.

    Raises:
        ValueError: If gold is empty.
    """
    if not gold:
        raise ValueError("no gold questions")
    recalls, ranks = [], []
    for question, relevant in gold:
        ranked = list(dict.fromkeys(search(question)))
        recalls.append(recall_at_k(ranked, relevant, k))
        ranks.append(reciprocal_rank(ranked, relevant))
    return {"recall_at_k": sum(recalls) / len(gold), "mrr": sum(ranks) / len(gold)}
```

1. **Line 10.** A question without gold labels would divide by zero. Fail fast.
2. **Line 14.** Divide by the number of relevant IDs, not by k.
3. **Line 19.** Ranks count from 1, so a first-place hit scores 1.0.
4. **Line 42.** Repeated sections keep their first position only.
5. **Line 45.** Two numbers to compare across chunk sizes, k and rankings.

Score retrieval apart from generation, on a versioned gold set, with recall at k and MRR, and compare settings on the same questions.

Recap card

- **Recall at k** len(set(ranked[:k]) & relevant) / len(relevant). With one relevant section per question, its mean is the hit rate.
- **Mean reciprocal rank** Mean over questions of 1 / rank of the first relevant result, ranks from 1, or 0 when none appears.
- **Gold set rules** Real questions, relevance marked by section, versioned, and every failed question added.
- **Repeated sections in a ranking** list(dict.fromkeys(section_ids)) keeps each at its first position.

Practice

Exercise 1. Multiple choice (level 1)

A third of your assistant's answers are wrong. On the same questions, recall at 5 is 0.68. Where do you work first?

- a) Retrieval: for about a third of the questions, the answer never reaches the prompt
- b) The answer prompt: add more rules about accuracy
- c) A larger model
- d) A higher max_tokens

Why?

- r1) The model can only answer from what it is sent, so a retrieval miss caps answer quality.
- r2) Retrieval code is easier to change than prompts.

Exercise 2. Predict the output (level 2)

What does this print?

```python
ranked = ["weighting#caps", "weighting#w14", "sampling#bases",
    "weighting#targets"]
relevant = {"weighting#w14", "weighting#targets"}


def recall_at_k(ranked, relevant, k):
    return len(set(ranked[:k]) & relevant) / len(relevant)


def reciprocal_rank(ranked, relevant):
    for rank, item in enumerate(ranked, start=1):
        if item in relevant:
            return 1 / rank
    return 0.0


print(recall_at_k(ranked, relevant, 2), recall_at_k(ranked, relevant, 4),
    reciprocal_rank(ranked, relevant))
```

Exercise 3. Fill the gap (level 2)

Keep each section at its first position, then count the relevant sections in the top k.

```python
ranked = list(dict._____(1)(section_ids))
hits = len(set(ranked[:k]) _____(2) relevant)
```

Exercise 4. Write the function (level 3)

Write `recall_at_k(ranked: list[str], relevant: set[str], k: int) -> float`: the share of `relevant` found in the first `k` items of `ranked`.

- Raise `ValueError` when `relevant` is empty or `k` is below 1.
- Count each relevant ID once, even if `ranked` repeats it.
- `k` may be larger than `len(ranked)`.

Your code must pass these checks:

- Divide the relevant IDs in the top k by the number of relevant IDs.
- Count each relevant ID once, and allow k beyond the end of the list.
- Raise ValueError for an empty relevant set or k below 1.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Share of the relevant IDs found in the first k of a ranking.
def recall_at_k(
    ranked: list[str],
    relevant: set[str],
    k: int,
) -> float: ...
```

Starter code:

```python
def recall_at_k(ranked, relevant, k):
    return 0.0
```

Exercise 5. Write the function (level 3)

Write `reciprocal_rank(ranked: list[str], relevant: set[str]) -> float`: `1 / rank` of the first item of `ranked` that is in `relevant`, with ranks counted from 1, or `0.0` when none is.

Your code must pass these checks:

- Return 1 / rank of the first relevant item, counting ranks from 1.
- Return 0.0 when no item is relevant, including for an empty ranking.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# 1 / rank of the first relevant ID, counting from 1, or 0.0 if none.
def reciprocal_rank(
    ranked: list[str],
    relevant: set[str],
) -> float: ...
```

Starter code:

```python
def reciprocal_rank(ranked, relevant):
```

```
    return 0.0
```

Exercise 6. Spot the bug (level 4)

Recall at 5 never goes above 0.2, even for questions whose one relevant section comes first. Tap the line responsible, then fix it.

```
def recall_at_k(ranked, relevant, k):
    if not relevant:
        raise ValueError("relevant is empty")
    if k < 1:
        raise ValueError("k must be at least 1")
    hits = set(ranked[:k]) & relevant
    return len(hits) / k
```

Write down the line number of the bug. Then write the fix.

Sources

- 45 of 50 has a 95% Wilson interval of 0.786 to 0.957. (verified: statsmodels proportion_confint(45, 50, method="wilson"), statsmodels 0.14.)
- The E2 code prints 0.5 1.0 0.5. (verified: Run on Python 3.11 and 3.13.)

B22-L2 Evaluate answers, exact checks, token F1 and LLM judges

Concept

Retrieval scores say whether the passage arrived. Answer checks say whether the reply used it well. Use the cheapest check that can decide, in this order.

1. Exact checks. Some outputs have one right value: a theme label, a number, yes or no. Normalise both sides, such as case and spaces, and compare. The citation check and the refusal sentence from lesson B21-L3 are exact checks too.

2. Reference overlap. For short factual answers, compare the reply's words with a reference answer. **Token F1** counts shared words. Precision is shared words over the reply's words, recall is shared words over the reference's words, and F1 is `2 * p * r / (p + r)`. It is fast, but "Weights are not capped at 5" scores 0.91 against "Weights are capped at 5". Use it to follow trends, not to judge one answer.

3. An LLM judge. For free text, a second model call grades the reply against a rubric, such as "Is every claim supported by the cited sources?"

and "Does it answer the question?" Have the judge return structured output (lesson B20-L2): a pass or fail per criterion, with a one-line reason. Grade one criterion per call. A single 1 to 10 score hides why an answer failed.

Judges have known biases. Zheng and colleagues (NeurIPS 2023) report **position bias**, a preference for the answer shown first, along with **verbosity bias**, a preference for longer answers, and **self-enhancement bias**, a preference for the judge's own outputs. When a judge compares two answers, run it twice with the order swapped, and call a tie when the two verdicts disagree.

Validate the judge. Before you trust it, have people grade a sample of answers, and measure how often the judge agrees, criterion by criterion. Check again whenever the judge's prompt or model changes.

Why it matters

A team compared two prompt versions with a judge that saw version A's answer first every time. A won 71% of comparisons, and the team shipped it. A rerun with the order swapped gave B 64% of the wins. Most of A's lead came from going first. Running each comparison in both orders, and counting disagreements as ties, would have shown the two prompts were close.

Worked example: weak code

```
1 def judge_pair(judge, question, answer_a, answer_b):
2     verdict = judge(question, answer_a, answer_b)
3     return "a" if verdict == "first" else "b"
```

1. **Line 2.** A is always first, so position bias counts as a win for A.

Worked example: strong code

```
1  import re
2  from collections import Counter
3  from collections.abc import Callable
4  from typing import Literal
5
6  Judge = Callable[[str, str, str], Literal["first", "second"]]
7  TOKEN = re.compile(r"[a-z0-9]+")
8
9
10 def token_f1(prediction: str, reference: str) -> float:
11     """Word-overlap F1 between a reply and a reference answer.
12
13     Args:
14         prediction: The reply.
15         reference: A person's reference answer.
16
```

```
    Returns:
        A value from 0 to 1. 0.0 when either side has no words.
    """
    pred, ref = TOKEN.findall(prediction.lower()),
TOKEN.findall(reference.lower())
    shared = sum((Counter(pred) & Counter(ref)).values())
    if shared == 0:
        return 0.0
    precision, recall = shared / len(pred), shared / len(ref)
    return 2 * precision * recall / (precision + recall)


def judge_pair(judge: Judge, question: str, answer_a: str, answer_b: str)
-> Literal["a", "b", "tie"]:
    """Pairwise verdict that cancels position bias.

    Args:
        judge: Returns "first" or "second" for the better answer.
        question: The question both answers address.
        answer_a: The first candidate.
        answer_b: The second candidate.

    Returns:
        "a" or "b" when both orders agree, otherwise "tie".
    """
    forward = judge(question, answer_a, answer_b)
    backward = judge(question, answer_b, answer_a)
    if forward == "first" and backward == "second":
        return "a"
    if forward == "second" and backward == "first":
        return "b"
    return "tie"
```

1. **Line 21.** Counter & keeps each word's smaller count, so repeated words are not double-counted.
2. **Line 22.** Guards the division when nothing overlaps or a side is empty.
3. **Line 41.** The same comparison with the order swapped.
4. **Line 46.** Verdicts that flip with the order are position bias, not preference.

Exact checks first, token F1 for trends, and a validated, structured judge for free text, run in both orders for pairs.

Recap card

- **Order of answer checks** Exact checks first, then token F1 for trends, then a validated LLM judge for free text.
- **Token F1** Shared words with Counter &; p = shared / reply words, r = shared / reference words, F1 = 2pr / (p + r).

- **LLM judge biases (Zheng et al., NeurIPS 2023)** Position, verbosity and self-enhancement bias. Swap the order for pairs and call disagreements ties.
- **Before trusting a judge** Compare it with human grades on a sample, criterion by criterion, and re-check after any prompt or model change.

Practice

Exercise 1. Multiple choice (level 1)

You evaluate the theme coder from lesson B20-L2, which returns one of Price, Service, Product or Other. People have coded 400 answers. Which check do you use?

- a) Exact match of the model's theme against the person's theme
- b) An LLM judge asked whether the theme is reasonable
- c) Token F1 between the theme and the answer text
- d) Read 20 answers and decide

Why?

- r1) The output has one right value, so the cheapest exact check decides it, with no judge to validate.
- r2) Judges cannot read single words.

Exercise 2. Predict the output (level 2)

What does this print?

```python
import re


def tokens(text):
    return re.findall(r"[a-z0-9]+", text.lower())


reply = tokens("Weights are not capped at 5")
reference = tokens("Weights are capped at 5")
shared = len(set(reply) & set(reference))
p, r = shared / len(reply), shared / len(reference)
print(shared, round(p, 2), round(r, 2), round(2 * p * r / (p + r), 3))
```

Exercise 3. Fill the gap (level 2)

Complete token F1 from precision and recall.

```python
precision = shared / len(_____(1))
recall = shared / len(ref)
f1 = 2 * precision * recall / (_____(2))
```

Exercise 4. Write the function (level 3)

Write `token_f1(prediction: str, reference: str) -> float`.

- Tokens are `re.findall(r"[a-z0-9]+", text.lower())`.
- Shared tokens count with repeats, up to the smaller count on each side: use `Counter`.
- Return `0.0` when nothing is shared, including when either side has no tokens.

What you need to know.

`Counter` from `collections` counts items. `&` between two counters keeps each key with the smaller count, and `.values()` gives the counts:

```python
from collections import Counter

a = Counter(["x", "x", "y"])
b = Counter(["x", "z"])
a & b                       # Counter({'x': 1})
sum((a & b).values())       # 1
```

Your code must pass these checks:

- Lower-case, keep [a-z0-9]+ tokens, and return 2pr / (p + r).
- Count a repeated word at most as often as it appears on the other side.
- Return 0.0 when nothing is shared, without dividing by zero.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
TOKEN = ...

# Word-overlap F1 between a reply and a reference answer.
def token_f1(prediction: str, reference: str) -> float: ...
```

Starter code:

```python
def token_f1(prediction, reference):
    return 0.0
```

Exercise 5. Write the function (level 3)

Write `judge_pair(judge, question, answer_a, answer_b) -> str`. `judge(question, first, second)` returns `"first"` or `"second"` for the better answer.

- Call the judge twice: with `answer_a` first, then with `answer_b` first.
- Return `"a"` when both calls prefer `answer_a`, `"b"` when both prefer `answer_b`, and `"tie"` otherwise.

Your code must pass these checks:

- Return a or b when both orders agree on the same answer.
- A judge that always picks one position must give a tie.
- Call the judge with A first, then with B first.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Pairwise verdict that cancels position bias.
def judge_pair(
    judge: Judge,
    question: str,
    answer_a: str,
    answer_b: str,
) -> Literal['a', 'b', 'tie']: ...
```

Starter code:

```
def judge_pair(judge, question, answer_a, answer_b):
    return "a" if judge(question, answer_a, answer_b) == "first" else "b"
```

Exercise 6. Spot the bug (level 4)

The judge replies with JSON such as `{"verdict": "fail", "reason": "would only pass with a citation"}`. The pass rate jumped to 98% after a prompt change that made reasons longer. Tap the line responsible, then fix it.

```
def parse_verdict(reply):
    text = reply.lower()
    if "pass" in text:
        return "pass"
    return "fail"
```

Write down the line number of the bug. Then write the fix.

Sources

- Zheng et al., Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena (NeurIPS 2023 Datasets and Benchmarks), examine position, verbosity and self-enhancement biases. (search-only: Search results for arXiv 2306.05685, November 2026. arxiv.org was not reachable from the build session.)
- The E2 code prints 5 0.83 1.0 0.909; Counter & keeps the minimum count per key. (verified: Run on Python 3.11 and 3.13.)

B22-L3 Regression evals and release gates for prompt changes

Concept

Every change to a prompt, model, chunk size or retrieval setting can fix some answers and break others. A **regression eval** runs one evaluation set on the current version, the **baseline**, and on the **candidate**, then decides whether the candidate may ship.

- **Freeze the set and split it.** Keep a development set for trying ideas and a test set you only use to decide. Tuning on the test set makes its score optimistic, as with any model (module B11).
- **Compare item by item.** Pass rates hide churn. A candidate at 84% against a baseline at 84% can have fixed 20 items and broken 20 others. Count the **fixed** items (fail to pass) and the **broken** ones (pass to fail), and read every broken one.
- **Repeat noisy items.** Replies vary between calls, so run each item three times and take the majority verdict: it passes when `sum(runs) > len(runs) / 2`. An item that passes one run in three is unstable. Flag it, whatever the majority says.
- **Write the gate before you run.** For example: every critical item passes, at most two items break, and the pass rate drops by no more than two points. Critical items include questions the sources cannot answer, where the only right reply is the refusal.
- **Log the run.** Record model, prompt version, eval set version, verdicts per item and token counts in MLflow (module B14) or the run log. A later regression can then be traced to the change that caused it.

```
fixed = sorted(q for q in baseline if not baseline[q] and candidate[q])
broken = sorted(q for q in baseline if baseline[q] and not candidate[q])
```

Both runs must cover the same items. A candidate that skipped a hard question would look better than it is, so check the keys match before you compare.

Why it matters

A prompt rewrite raised the pass rate from 86% to 89%, and it shipped. A week later, clients received confident answers about wave W15, which had not started: the new prompt had stopped refusing questions the sources could not answer. Three critical "must refuse" items had broken, hidden

inside a higher average. A gate that required every critical item to pass would have blocked the release.

Worked example: weak code

```
1 def should_ship(baseline, candidate):
1     return sum(candidate.values()) >= sum(baseline.values())
```

1. **Line 2.** Only totals. Broken critical items and churn stay hidden, and mismatched items go unnoticed.

Worked example: strong code

```
1  def compare_runs(baseline: dict[str, bool], candidate: dict[str, bool]) -
   > dict[str, list[str]]:
2      """Items fixed and broken by the candidate.
3
4      Raises:
5          ValueError: If the two runs cover different items.
6      """
1      if baseline.keys() != candidate.keys():
8          raise ValueError("baseline and candidate cover different items")
9      return {
10         "fixed": sorted(q for q in baseline if not baseline[q] and
   candidate[q]),
2          "broken": sorted(q for q in baseline if baseline[q] and not
   candidate[q]),
12     }
13
14
15 def release_gate(
16     baseline: dict[str, bool],
17     candidate: dict[str, bool],
18     critical: set[str],
19     max_broken: int = 2,
20     max_drop: float = 0.02,
21 ) -> tuple[bool, list[str]]:
22     """Decide whether the candidate may ship, with the reasons if not.
23
24     Args:
25         baseline: Pass or fail per item for the current version.
26         candidate: Pass or fail per item for the new version.
27         critical: Items that must pass, such as must-refuse questions.
28         max_broken: Most items allowed to go from pass to fail.
29         max_drop: Largest allowed fall in pass rate, as a share.
30
31     Returns:
32         True with no reasons, or False with every rule that failed.
33     """
34     changes = compare_runs(baseline, candidate)
3      reasons = [f"critical:{q}" for q in sorted(critical) if not
   candidate[q]]
```

```
    if len(changes["broken"]) > max_broken:
        reasons.append("too_many_broken")
    drop = sum(baseline.values()) / len(baseline) -
sum(candidate.values()) / len(candidate)
    if drop > max_drop:
        reasons.append("pass_rate_drop")
    return not reasons, reasons
```

1. **Line 7.** Both runs must cover the same items.
2. **Line 11.** Pass to fail: read every one of these.
3. **Line 35.** Critical items must pass in the candidate, whatever the average.
4. **Line 41.** The decision comes with every reason, for the release note.

Compare per item, repeat noisy items, write the gate first, and never let a higher average hide a broken critical item.

Recap card

- **Fixed and broken items** Fixed: fail in the baseline, pass in the candidate. Broken: the reverse. Read every broken item.
- **Majority over repeated runs** Pass when sum(runs) > len(runs) / 2. Unstable when 0 < sum(runs) < len(runs).
- **A release gate** Written before the run: every critical item passes, at most N items break, and the pass rate drops by no more than a set share.
- **What to log for each eval run** Model, prompt version, eval set version, verdicts per item and token counts.

Practice

Exercise 1. Multiple choice (level 1)

On the test set, the baseline and the candidate prompt both pass 84% of items. What do you do before deciding?

- a) Count the fixed and broken items and read every broken one
- b) Ship the candidate: it is no worse
- c) Keep the baseline: it is no worse
- d) Tune the candidate on the test set until it passes 90%

Why?

- r1) Equal pass rates can hide many fixes and as many breaks, including critical ones.
- r2) Reading items is faster than computing a pass rate.

Exercise 2. Predict the output (level 2)

What does this print?

```
baseline = {"q1": True, "q2": True, "q3": False, "q4": False, "q5": True}
candidate = {"q1": True, "q2": False, "q3": True, "q4": True, "q5": True}
fixed = sorted(q for q in baseline if not baseline[q] and candidate[q])
broken = sorted(q for q in baseline if baseline[q] and not candidate[q])
print(sum(baseline.values()), sum(candidate.values()), fixed, broken)
```

Exercise 3. Fill the gap (level 2)

Three runs per item: take the majority verdict, and flag items whose runs disagree.

```
passed = sum(runs) _____(1) len(runs) / 2
unstable = 0 < sum(runs) < _____(2)(runs)
```

Exercise 4. Write the function (level 3)

Write `compare_runs(baseline: dict[str, bool], candidate: dict[str, bool]) -> dict[str, list[str]]`.

- Raise `ValueError` when the two dicts have different keys.
- Return `{"fixed": [...], "broken": [...]}`: items that went from fail to pass, and from pass to fail, each sorted.

Your code must pass these checks:

- Fixed is fail to pass and broken is pass to fail, each sorted.
- Unchanged items are neither fixed nor broken.
- Raise ValueError when the two runs cover different items.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Items fixed and broken by the candidate.
def compare_runs(
    baseline: dict[str, bool],
    candidate: dict[str, bool],
) -> dict[str, list[str]]: ...
```

Starter code:

```
def compare_runs(baseline, candidate):
    return {"fixed": [], "broken": []}
```

Exercise 5. Write the function (level 3)

Write `release_gate(baseline, candidate, critical, max_broken=2, max_drop=0.02) -> tuple[bool, list[str]]`. `compare_runs` is defined for you.

- Add `"critical:<id>"` for each critical item that fails in the candidate, in sorted order.
- Add `"too_many_broken"` when more than `max_broken` items broke.
- Add `"pass_rate_drop"` when the candidate's pass rate is more than `max_drop` below the baseline's.
- Return `(True, [])` when no rule fails, otherwise `(False, reasons)`.

Your code must pass these checks:

- Return (True, []) when every rule passes.
- Fail every critical item that fails in the candidate, even if it failed before too.
- Add too_many_broken above max_broken, and pass_rate_drop when the rate falls by more than max_drop.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Decide whether the candidate may ship, with the reasons if not.
def release_gate(
    baseline: dict[str, bool],
    candidate: dict[str, bool],
    critical: set[str],
    max_broken: int = 2,
    max_drop: float = 0.02,
) -> tuple[bool, list[str]]: ...
```

Starter code:

```
def release_gate(baseline, candidate, critical, max_broken=2,
    max_drop=0.02):
    return sum(candidate.values()) >= sum(baseline.values()), []
```

Exercise 6. Spot the bug (level 4)

A candidate prompt stopped refusing questions the sources cannot answer, and the gate let it ship. Tap the line responsible, then fix it.

```
def release_gate(baseline, candidate, critical, max_broken=2,
    max_drop=0.02):
    reasons = []
    changes = compare_runs(baseline, candidate)
    for item in sorted(critical):
        if not baseline[item]:
            reasons.append(f"critical:{item}")
    if len(changes["broken"]) > max_broken:
```

```
        reasons.append("too_many_broken")
    drop = sum(baseline.values()) / len(baseline) -
sum(candidate.values()) / len(candidate)
    if drop > max_drop:
        reasons.append("pass_rate_drop")
    return not reasons, reasons
```

Write down the line number of the bug. Then write the fix.

Sources

- The E2 code prints 3 4 ['q3', 'q4'] ['q2']. (verified: Run on Python 3.11 and 3.13.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B22-L1 Evaluate retrieval, recall at k and MRR on a gold set

B22-L1 exercise 1

Hints. (1) What does recall at 5 of 0.68 mean for the prompt? (2) Can the model answer from a passage it never saw? (3) Fix the step that caps the rest.

Answer. a) Retrieval: for about a third of the questions, the answer never reaches the prompt Because: r1) The model can only answer from what it is sent, so a retrieval miss caps answer quality.

With recall at 5 of 0.68, roughly a third of prompts lack the answer. Fix retrieval first, then measure answers again.

Common wrong answers.

- `b`: No prompt helps when the right section is missing from the sources.
- `c`: A larger model still cannot cite a passage it was never given.
- `d`: max_tokens limits the reply length. It does not change which sources arrive.
- `a|r2`: Right place, wrong reason. Retrieval caps what the model can answer.

B22-L1 exercise 2

Hints. (1) How many relevant IDs are in the first two items? (2) And in all four? (3) Where is the first relevant item?

Answer.

```
1 0.5 1.0 0.5
```

The top 2 hold one of two relevant sections (0.5), the top 4 hold both (1.0), and the first hit is at rank 2 (0.5).

Common wrong answers.

- `0.5 1.0 1.0`: The first item, weighting#caps, is not relevant. The first hit is at rank 2.

- `0.5 0.5 0.5`: Recall divides by the number of relevant IDs, 2. Both appear in the top 4.
- `1.0 1.0 0.5`: Only one of the two relevant IDs is in the top 2.

B22-L1 exercise 3

Hints. (1) Which dict method builds a dict from a sequence of keys? (2) Which set operator keeps only shared items? (3) `fromkeys` and `&`.

Answer.

```
ranked = list(dict.fromkeys(section_ids))
hits = len(set(ranked[:k]) & relevant)
```

`dict.fromkeys` drops repeats and keeps first positions. `&` keeps the sections that are both retrieved and relevant.

Common wrong answers.

- g1 = `fromkeys`; g2 = `|`: | is the union. The hits are the intersection: &.

B22-L1 exercise 4

Hints. (1) Guard the two bad inputs first. (2) `set(ranked[:k]) & relevant` removes repeats. (3) Divide by `len(relevant)`.

Answer.

```
def recall_at_k(ranked: list[str], relevant: set[str], k: int) -> float:
    """Share of the relevant IDs found in the first k of a ranking.

    Args:
        ranked: Retrieved IDs, best first.
        relevant: IDs a person marked as answering the question.
        k: Cut-off.

    Returns:
        A value from 0 to 1.

    Raises:
        ValueError: If relevant is empty or k is below 1.
    """
    if not relevant:
        raise ValueError("relevant is empty: the question has no gold
answer")
    if k < 1:
        raise ValueError(f"k must be at least 1, got {k}")
    return len(set(ranked[:k]) & relevant) / len(relevant)
```

The set intersection counts each relevant ID once, and dividing by the number of relevant IDs gives recall. Dividing by k would give precision.

B22-L1 exercise 5

Hints. (1) `enumerate(ranked, start=1)` gives ranks from 1. (2) Return as soon as you find a relevant item. (3) After the loop, return 0.0.

Answer.

```
def reciprocal_rank(ranked: list[str], relevant: set[str]) -> float:
    """1 / rank of the first relevant ID, counting from 1, or 0.0 if none.

    Args:
        ranked: Retrieved IDs, best first.
        relevant: IDs a person marked as answering the question.

    Returns:
        A value from 0 to 1.
    """
    for rank, item in enumerate(ranked, start=1):
        if item in relevant:
            return 1 / rank
    return 0.0
```

Returning at the first hit gives 1 / rank of the best-placed relevant item. A ranking with no hit scores 0.

B22-L1 exercise 6

Hints. (1) With one relevant section and k = 5, what is the largest value? (2) Recall is a share of what? (3) Divide by len(relevant).

Answer. The bug is on line 7.

Fixed version:

```
def recall_at_k(ranked, relevant, k):
    if not relevant:
        raise ValueError("relevant is empty")
    if k < 1:
        raise ValueError("k must be at least 1")
    hits = set(ranked[:k]) & relevant
    return len(hits) / len(relevant)
```

Dividing by k gives precision at k, which cannot exceed 1/5 when a question has one relevant section. Recall divides by the number of relevant sections.

Common wrong answers.

- Line 6: The intersection finds the right hits. Look at what they are divided by.
- Line 2: The guard is right. It stops a division by zero.

B22-L2 Evaluate answers, exact checks, token F1 and LLM judges

B22-L2 exercise 1

Hints. (1) How many right values does each answer have? (2) Which check needs no model? (3) Compare labels directly.

Answer. a) Exact match of the model's theme against the person's theme Because: r1) The output has one right value, so the cheapest exact check decides it, with no judge to validate.

A closed label has one right value. Compare it exactly with the human label and report agreement.

Common wrong answers.

- `b`: A judge adds cost and its own errors. With a person's label to compare against, an exact match decides.
- `c`: Token F1 compares two texts that should say the same thing. A theme and an answer do not.
- `d`: You already have 400 labelled answers. Use all of them, with an exact check.
- `a|r2`: Right check, wrong reason. Judges can grade labels. Exact match is cheaper and exact.

B22-L2 exercise 2

Hints. (1) Count the words in each list. (2) Which reply word is not in the reference? (3) p = 5/6, r = 5/5.

Answer.

```
1 5 0.83 1.0 0.909
```

The reply shares all five reference words and adds "not". F1 is 0.909, although the reply says the opposite. Overlap measures words, not meaning.

Common wrong answers.

- `5 1.0 0.83 0.909`: Precision divides by the reply's 6 words. Recall divides by the reference's 5.
- `4 0.67 0.8 0.727`: All five reference words appear in the reply. Only "not" is extra.

B22-L2 exercise 3

Hints. (1) Precision asks: how much of the reply is right? (2) F1 is the harmonic mean of precision and recall. (3) `pred` and `precision + recall`.

Answer.

```
precision = shared / len(pred)
recall = shared / len(ref)
f1 = 2 * precision * recall / (precision + recall)
```

Precision divides by the reply's words, recall by the reference's, and F1 is `2pr / (p + r)`.

Common wrong answers.

- g1 = `ref`; g2 = `precision + recall`: Precision divides by the reply's words, pred. Recall uses the reference.

B22-L2 exercise 4

Hints. (1) Tokenise both sides with `re.findall` on the lower-cased text. (2) `sum((Counter(pred) & Counter(ref)).values())` counts shared tokens. (3) Return 0.0 before dividing when nothing is shared.

Answer.

```
import re
from collections import Counter

TOKEN = re.compile(r"[a-z0-9]+")


def token_f1(prediction: str, reference: str) -> float:
    """Word-overlap F1 between a reply and a reference answer.

    Args:
        prediction: The reply.
        reference: A person's reference answer.

    Returns:
        A value from 0 to 1. 0.0 when either side has no words.
    """
    pred, ref = TOKEN.findall(prediction.lower()),
    TOKEN.findall(reference.lower())
    shared = sum((Counter(pred) & Counter(ref)).values())
    if shared == 0:
        return 0.0
    precision, recall = shared / len(pred), shared / len(ref)
    return 2 * precision * recall / (precision + recall)
```

`Counter &` caps each shared word at its smaller count, and the zero guard covers empty and disjoint texts before any division.

B22-L2 exercise 5

Hints. (1) Call the judge once in each order. (2) In the swapped call, preferring A means the verdict is "second". (3) Anything else is a tie.

Answer.

```python
from collections.abc import Callable
from typing import Literal

Judge = Callable[[str, str, str], Literal["first", "second"]]


def judge_pair(judge: Judge, question: str, answer_a: str, answer_b: str)
    -> Literal["a", "b", "tie"]:
    """Pairwise verdict that cancels position bias.

    Args:
        judge: Returns "first" or "second" for the better answer.
        question: The question both answers address.
        answer_a: The first candidate.
        answer_b: The second candidate.

    Returns:
        "a" or "b" when both orders agree, otherwise "tie".
    """
    forward = judge(question, answer_a, answer_b)
    backward = judge(question, answer_b, answer_a)
    if forward == "first" and backward == "second":
        return "a"
    if forward == "second" and backward == "first":
        return "b"
    return "tie"
```

In the swapped call, A sits second. A real preference for A gives "first" then "second". A judge that always picks one position gives the same word twice, which is a tie.

B22-L2 exercise 6

Hints. (1) Where can the word pass appear in a failing reply? (2) The reply is JSON with a verdict field. (3) json.loads(reply)["verdict"].

Answer. The bug is on line 3.

Fixed version:

```python
import json


def parse_verdict(reply):
    verdict = json.loads(reply)["verdict"]
    if verdict not in {"pass", "fail"}:
        raise ValueError(f"unknown verdict: {verdict}")
```

```
8     return verdict
```

"pass" appears inside the reason of a failing verdict, so the substring test counts it as a pass. Parse the JSON and read the verdict field, and reject anything that is not pass or fail.

Common wrong answers.

- Line 2: Lower-casing is harmless. The test on the next line looks at the wrong text.
- Line 5: Defaulting to fail is the safer side. The pass branch fires too often.

B22-L3 Regression evals and release gates for prompt changes

B22-L3 exercise 1

Hints. (1) Can two runs with the same total differ? (2) Which items matter most? (3) Fixed and broken.

Answer. a) Count the fixed and broken items and read every broken one Because: r1) Equal pass rates can hide many fixes and as many breaks, including critical ones.

Equal totals say nothing about which items changed. Count fixed and broken items, and read the broken ones.

Common wrong answers.

- `b`: Equal totals can hide broken critical items. Look per item first.
- `c`: The candidate may fix important items. You cannot tell from the totals.
- `d`: Tuning on the test set makes its score optimistic. Tune on the development set.
- `a|r2`: Right step, wrong reason. It is about churn hidden by equal totals.

B22-L3 exercise 2

Hints. (1) sum counts True as 1. (2) Which items went from False to True? (3) And from True to False?

Answer.

```
1 3 4 ['q3', 'q4'] ['q2']
```

The candidate passes one more item overall, but it fixed two and broke one. The broken item, q2, needs reading before any release.

Common wrong answers.

- `3 4 ['q3', 'q4'] []`: q2 passed in the baseline and fails in the candidate. It is broken.
- `3 4 ['q2'] ['q3', 'q4']`: Fixed means fail to pass. q3 and q4 went from False to True.

B22-L3 exercise 3

Hints. (1) A majority is more than half. (2) Unstable: some runs pass, but not all. (3) `>` and `len`.

Answer.

```
passed = sum(runs) > len(runs) / 2
unstable = 0 < sum(runs) < len(runs)
```

A majority needs more than half the runs to pass. An item is unstable when the pass count is above 0 and below the number of runs.

Common wrong answers.

- g1 = `>=`; g2 = `len`: With an even number of runs, >= would pass a split vote. A majority is strictly more than half.

B22-L3 exercise 4

Hints. (1) Compare `baseline.keys()` with `candidate.keys()` first. (2) Fixed: `not baseline[q] and candidate[q]`. (3) Wrap each comprehension in `sorted`.

Answer.

```
def compare_runs(baseline: dict[str, bool], candidate: dict[str, bool]) -> dict[str, list[str]]:
    """Items fixed and broken by the candidate.

    Args:
        baseline: Pass or fail per item for the current version.
        candidate: Pass or fail per item for the new version.

    Returns:
        Sorted item IDs under "fixed" (fail to pass) and "broken" (pass to fail).

    Raises:
        ValueError: If the two runs cover different items.
    """
    if baseline.keys() != candidate.keys():
        raise ValueError("baseline and candidate cover different items")
    return {
        "fixed": sorted(q for q in baseline if not baseline[q] and candidate[q]),
```

```
        "broken": sorted(q for q in baseline if baseline[q] and not
    candidate[q]),
    }
```

The key check stops a run with missing items from looking better than it is. The two comprehensions split the changes by direction.

B22-L3 exercise 5

Hints. (1) Start the reasons with the critical items that fail in the candidate. (2) Use `compare_runs` for the broken count. (3) Pass rate is `sum(run.values()) / len(run)`.

Answer.

```
def release_gate(
    baseline: dict[str, bool],
    candidate: dict[str, bool],
    critical: set[str],
    max_broken: int = 2,
    max_drop: float = 0.02,
) -> tuple[bool, list[str]]:
    """Decide whether the candidate may ship, with the reasons if not.

    Args:
        baseline: Pass or fail per item for the current version.
        candidate: Pass or fail per item for the new version.
        critical: Items that must pass, such as must-refuse questions.
        max_broken: Most items allowed to go from pass to fail.
        max_drop: Largest allowed fall in pass rate, as a share.

    Returns:
        True with no reasons, or False with every rule that failed.
    """
    changes = compare_runs(baseline, candidate)
    reasons = [f"critical:{q}" for q in sorted(critical) if not
    candidate[q]]
    if len(changes["broken"]) > max_broken:
        reasons.append("too_many_broken")
    drop = sum(baseline.values()) / len(baseline) -
    sum(candidate.values()) / len(candidate)
    if drop > max_drop:
        reasons.append("pass_rate_drop")
    return not reasons, reasons
```

Each rule adds its own reason, so a blocked release says exactly why. Critical items are judged on the candidate alone: a must-refuse item that failed before still blocks.

B22-L3 exercise 6

Hints. (1) Which version is about to ship? (2) Whose verdicts should the critical rule read? (3) candidate[item].

Answer. The bug is on line 5.

Fixed version:

```
def release_gate(baseline, candidate, critical, max_broken=2,
    max_drop=0.02):
    reasons = []
    changes = compare_runs(baseline, candidate)
    for item in sorted(critical):
        if not candidate[item]:
            reasons.append(f"critical:{item}")
    if len(changes["broken"]) > max_broken:
        reasons.append("too_many_broken")
    drop = sum(baseline.values()) / len(baseline) -
    sum(candidate.values()) / len(candidate)
    if drop > max_drop:
        reasons.append("pass_rate_drop")
    return not reasons, reasons
```

The critical rule reads the baseline, which still refused correctly, so the candidate's failures never count. Read `candidate[item]`.

Common wrong answers.

- Line 7: One broken critical item is within max_broken, so this rule cannot catch it alone. The critical rule must.
- Line 9: The pass rate barely moves for one item. The critical check looks at the wrong run.